NATURAL LANGUAGE PROCESSING FUNDAMENTALS

From Text Preprocessing to Word Embeddings, with Python

Draft manuscript — Part IV: Linguistic Analysis with spaCy

Chapters 10–13 · spaCy Pipelines, Part-of-Speech Tagging, Dependency Parsing, Named Entity Recognition

[Author name — to be supplied]

Contents

Part IV Linguistic Analysis with spaCy 3
Chapter 10 spaCy Pipelines and Architecture 4
10.1 What Is a spaCy Pipeline? 4
10.2 What Happens When We Process Text? 5
10.3 Blank, Pre-trained and Custom Pipelines 5
10.4 Inspecting a Pipeline 7
10.5 The Main Pipeline Components 8
10.6 Modular Pipelines and Custom Components 10
10.7 Pipeline Order Matters 11
10.8 Multilingual Pipelines 12
10.9 Library, Language Object, Pipeline, Component and Model 13
10.10 Choosing a Pipeline 13
10.11 Common Mistakes 13
10.12 Chapter Summary 14
10.13 Review Questions and Exercises 14
Chapter 11 Part-of-Speech Tagging 15
11.1 What Is POS Tagging? 15
11.2 Types of Parts of Speech 15
11.3 POS Tagging with spaCy 16
11.4 Coarse and Fine-Grained Tags: pos_ and tag_ 17
11.5 Understanding spacy.explain() 19
11.6 Filtering Tokens by POS 19
11.7 Counting POS Tags with count_by() 20
11.8 Practical Applications of POS Tagging 21
11.9 Context Decides the Tag 21
11.10 Common Mistakes 21
11.11 Chapter Summary 22
11.12 Review Questions and Exercises 22
Chapter 12 Dependency Parsing 23
12.1 What Is Dependency Parsing? 23
12.2 Dependency Parsing with spaCy 23
12.3 Dependency Labels 24
12.4 Head, Children and Subtree 24
12.5 Visualizing a Parse with displaCy 26
12.6 Extracting Subject–Verb–Object Relations 26
12.7 Parser-Based Sentence Segmentation 27
12.8 When the Parse Is Wrong 27
12.9 Common Mistakes 28
12.10 Chapter Summary 28
12.11 Review Questions and Exercises 28
Chapter 13 Named Entity Recognition 30
13.1 What Is Named Entity Recognition? 30
13.2 Types of Named Entities 30
13.3 Why Is NER Useful? 32
13.4 Implementing NER with spaCy 32
13.5 Visualizing Entities with displaCy 33
13.6 Custom Entities with Span and set_ents() 34
13.7 Three Approaches to Building an NER System 35
13.8 Comparing the Approaches 38
13.9 Evaluating NER 39
13.10 NER on the Chapter 5 Biography 39
13.11 Common Mistakes 40
13.12 Chapter Summary 40
13.13 Review Questions and Exercises 40

Part IV Linguistic Analysis with spaCy

Part III prepared text for analysis: cleaning it, splitting it into sentences and tokens, deciding which words to drop and reducing words to base forms. Part IV turns to what a trained pipeline can tell us about the text itself. It begins with the architecture of a spaCy pipeline (Chapter 10) and then examines three of its components in turn: part-of-speech tagging (Chapter 11), dependency parsing (Chapter 12) and named entity recognition (Chapter 13).

Throughout Part IV we use one pre-trained English model, and every output shown was produced with it. Where the output differs from what the original study notes expected, both are shown and labelled. The point of these chapters is not that the model is always right; it is to learn what each component produces, how to read it, and where to be careful. Part V then moves from analysing text to representing it as numbers.

All outputs in Part IV were produced with Python 3.12.3, spaCy 3.8.16 and the English model `en_core_web_sm` 3.8.0. Other versions, other models and larger models can behave differently, so treat model-specific results as observations rather than rules. Section 3.3 explains how to set up the environment.

Chapter 10 spaCy Pipelines and Architecture

Learning objectives

After studying this chapter you will be able to:

- explain what a spaCy pipeline is and how the tokenizer differs from pipeline components;
- distinguish blank, pre-trained and custom pipelines, and choose among them;
- inspect a pipeline with `pipe_names` and `pipeline`;
- describe the role of the tagger, lemmatizer, parser, NER component and sentencizer;
- build a custom pipeline with `add_pipe()` and explain why component order matters;
- separate the terms library, language object, pipeline, component and model.

Part III used spaCy mainly for its tokenizer. This chapter opens Part IV by looking at the whole machine: the pipeline that turns raw text into a richly annotated `Doc`. Once the architecture is clear, Chapters 11 to 13 examine three of its components in detail: part-of-speech tagging, dependency parsing and named entity recognition.

10.1 What Is a spaCy Pipeline?

In spaCy, a **pipeline** is a sequence of processing components that analyse text and add linguistic information to a `Doc` object. When raw text is passed to spaCy, the **tokenizer** runs first and breaks the text into tokens (Chapter 7). After tokenization, further components can add information such as part-of-speech tags, lemmas, syntactic dependencies, named entities, sentence boundaries and custom annotations.

The tokenizer and the pipeline components are related, but they are not the same thing. **The tokenizer creates the tokens; the pipeline components analyse those tokens.** The tokenizer builds the basic structure of the document, and the components add linguistic information to it. Figure 10.1 shows the two steps for the English pre-trained pipeline used in this book.

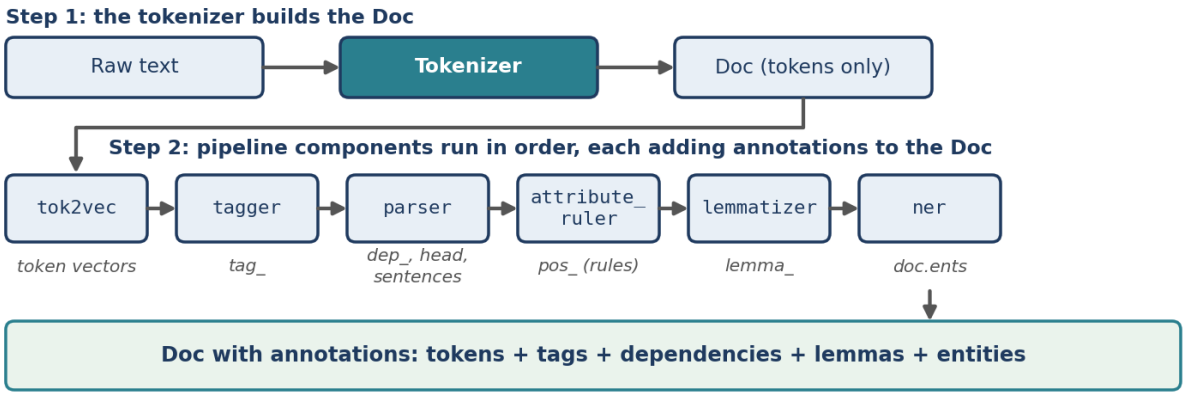


Figure 10.1 The tokenizer builds the Doc; the pipeline components then run in order and add annotations. The components shown are those of en_core_web_sm 3.8.0.

10.2 What Happens When We Process Text?

Consider the sentence “Apple is opening a new office in Chennai.” The single call `nlp(text)` performs three steps. First, the text is divided into tokens. Second, the components of the loaded pipeline process those tokens: tagging identifies grammatical categories, lemmatization finds base forms, dependency parsing identifies grammatical relationships, and NER identifies entities such as organisations and places. Third, the results are stored in the `Doc`, which holds the original text together with every annotation.

```
import spacy

nlp = spacy.load("en_core_web_sm")
doc = nlp("Apple is opening a new office in Chennai.")

print([token.text for token in doc])
for token in doc:
    print(token.text, token.pos_, token.dep_, token.lemma_)
print([(ent.text, ent.label_) for ent in doc.ents])
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
['Apple', 'is', 'opening', 'a', 'new', 'office', 'in', 'Chennai', '.']
Apple PROPN nsubj Apple
is AUX aux be
opening VERB ROOT open
a DET det a
new ADJ amod new
office NOUN dobj office
in ADP prep in
Chennai PROPN pobj Chennai
. PUNCT punct .
[('Apple', 'ORG'), ('Chennai', 'GPE')]
```

With a trained pipeline loaded, `nlp(text)` therefore does much more than split text into tokens. Each token now carries a part-of-speech tag (`pos_`), a dependency label (`dep_`) and a lemma (`lemma_`), and the `Doc` carries its entities. Chapters 11 to 13 explain these attributes one by one.

10.3 Blank, Pre-trained and Custom Pipelines

There are three useful ways to think about spaCy pipelines (Figure 10.2).

Blank pipeline

A **blank pipeline** is a new language object without a trained statistical model. For English it is created with `spacy.blank("en")`, and it primarily gives you the language-specific tokenizer. It does not automatically provide POS tagging, named entity recognition, dependency parsing or statistical lemmatization unless the required components are explicitly added and configured. A blank pipeline is a lightweight starting point that gives you more control, which is useful for custom NLP systems, for experimenting with individual components, and for avoiding the loading of components you will never use.

```
blank = spacy.blank("en")
doc = blank("I am learning NLP.")
```

```
print(blank.pipe_names)
print([token.text for token in doc])
print([token.pos_ for token in doc])
print([token.dep_ for token in doc])
print(doc.ents)
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
[]
['I', 'am', 'learning', 'NLP', '.']
['', '', '', '', '']
['', '', '', '', '']
()
```

The blank pipeline has no components, so the part-of-speech and dependency attributes are empty strings and there are no entities.

Pre-trained pipeline

A **pre-trained pipeline** contains trained components that can analyse text immediately. It is loaded with `spacy.load("en_core_web_sm")`, and its components have already been trained on language data, so you do not need to build and train each one yourself. The exact components depend on the model and the spaCy version.

Installing spaCy and installing a model are separate steps. Installing the library gives you the NLP framework; a model such as `en_core_web_sm` provides the trained linguistic capabilities:

```
pip install spacy
python -m spacy download en_core_web_sm
```

Custom pipeline

A **custom pipeline** starts from `spacy.blank("en")` and adds only the components the application needs, for example the tokenizer plus NER only. Section 10.6 builds one.

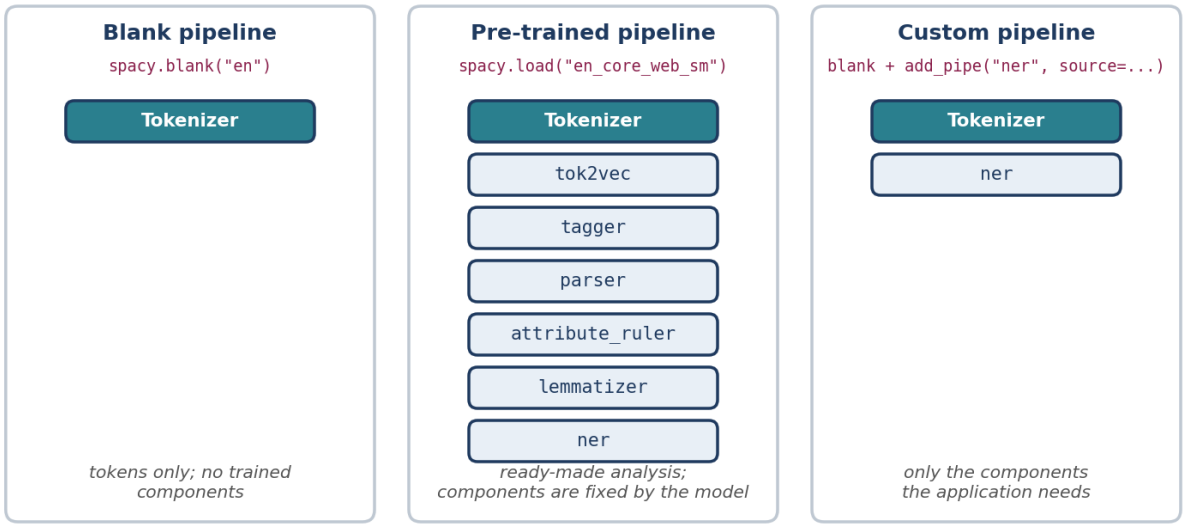


Figure 10.2 Blank, pre-trained and custom pipelines. In every case the tokenizer is handled separately from the list of components.

Feature	Blank pipeline	Pre-trained pipeline
Created with	`spacy.blank()`	`spacy.load()`
Tokenizer	Yes	Yes
Trained NLP components	No by default	Yes
POS tagging	Not automatically	Available if the model contains a tagger
NER	Not automatically	Available if the model contains NER
Dependency parsing	Not automatically	Available if the model contains a parser
Starting point	Custom NLP systems	Ready-to-use NLP
Control	High	Less manual setup
Typical use	Customisation or lightweight processing	General NLP applications

Table 10.1 Blank pipeline compared with pre-trained pipeline.

A simple memory rule: `blank()` starts from scratch; `load()` uses an existing trained pipeline.

10.4 Inspecting a Pipeline

Two properties show what is currently active. `nlp.pipe_names` lists the component names and `nlp.pipeline` lists the names together with the component objects. Inspection answers practical questions: which components are loaded, whether NER, a tagger or a parser is available, and whether unnecessary components are being loaded.

```
print(nlp.pipe_names)
print([(name, type(comp).__name__) for name, comp in nlp.pipeline])
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
['tok2vec', 'tagger', 'parser', 'attribute_ruler', 'lemmatizer', 'ner']
[('tok2vec', 'Tok2Vec'), ('tagger', 'Tagger'), ('parser',
'DependencyParser'), ('attribute_ruler', 'AttributeRuler'), ('lemmatizer',
'EnglishLemmatizer'), ('ner', 'EntityRecognizer')]
```

The notes show the following as an example of what `nlp.pipe_names` may print, and add that the exact list differs between models and spaCy versions:

Output given in the notes (not executed here)

```
['tok2vec', 'tagger', 'parser', 'ner', 'lemmatizer']
```

The verified list differs from the notes' example in two ways: it contains an additional `attribute_ruler` component, and `lemmatizer` comes before `ner`. This is consistent with the notes' own caution that the list depends on the model and version, and the difference matters for the next sections because the order of components is part of the pipeline's behaviour. Section 9.4 introduced the `attribute_ruler`.

10.5 The Main Pipeline Components

A trained pipeline can contain several components. Table 10.2 lists the most important ones and where this book treats each in detail.

Component	Main purpose	Where it is covered
Tokenizer	Splits raw text into tokens	Chapter 7
Tagger	Assigns part-of-speech information	Chapter 11
Lemmatizer	Determines base or dictionary forms	Chapter 9
Parser	Determines syntactic relationships	Chapter 12
NER	Identifies named entities	Chapter 13
Sentencizer	Determines sentence boundaries with rules	Section 7.8
Custom components	Perform application-specific processing	Section 10.6

Table 10.2 Major spaCy pipeline components.

Tagger

The tagger assigns grammatical information to tokens. POS stands for part of speech. In “The dog runs quickly.” the words play different grammatical roles: *The* is a determiner, *dog* a noun, *runs* a verb and *quickly* an adverb. POS information helps applications identify nouns and verbs, analyse sentence structure and support information extraction. Chapter 11 treats this component in full.

```
doc = nlp("The dog runs quickly.")
for token in doc:
    print(token.text, token.pos_)
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
The DET
dog NOUN
runs VERB
quickly ADV
. PUNCT
```

This matches the “possible output” given in the notes. The notes add that the exact tags and output can depend on the model.

Lemmatizer

A lemmatizer determines the base or dictionary form of a word, its **lemma**: *running* becomes *run*, *played* becomes *play* and *studies* becomes *study*. Unlike a stemmer, which may produce *studi*, the lemmatizer returns linguistically meaningful base forms, and it can use other linguistic information to choose an appropriate form. Section 9.3 explains lemmatization in detail; here we only observe the component in the pipeline.

```
doc = nlp("The students are studying.")
for token in doc:
    print(token.text, token.lemma_)
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
The the
students student
are be
studying study
. .
```

The notes' conceptual result (the, student, be, study) matches the verified lemmas.

Parser

The dependency parser analyses grammatical relationships between words. In "Mani reads a book." *Mani* is the subject of *reads* and *book* is its object. This lets spaCy represent the structure of a sentence, which supports relationship extraction, question answering and information extraction. Chapter 12 treats the parser in full.

```
doc = nlp("Mani reads a book.")
for token in doc:
    print(token.text, token.dep_, token.head.text)
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
Mani nsubj reads
reads ROOT reads
a det book
book dobj reads
. punct reads
```

The notes describe this conceptually (subject, root, object); the real output adds the determiner *a* and the full stop, and uses spaCy's own label names such as `nsubj` and `dobj`. Chapter 12 explains them.

Named entity recogniser

Named entity recognition identifies real-world entities such as organisations, places and amounts of money. It is not a search for predefined words: a trained NER model uses learned language patterns to decide whether a piece of text is an entity, and the entity labels depend on the model. Chapter 13 treats NER in full.

```
doc = nlp("Apple opened an office in Chennai.")
for ent in doc.ents:
    print(ent.text, ent.label_)
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
Apple ORG
Chennai GPE
```

This matches the notes' possible output (Apple ORG, Chennai GPE). The notes also use a longer sentence, "Apple opened a new office in Chennai for ₹50 crore", and expect an organisation, a location and an amount of money:

```
doc = nlp("Apple opened a new office in Chennai for ₹50 crore.")
for ent in doc.ents:
```

```
    print(ent.text, ent.label_)
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
Apple ORG
Chennai ORG
50 MONEY
```

The verified output differs from that expectation: *Chennai* is labelled `ORG` here but `GPE` in the shorter sentence, and the amount is returned as `50` with the label `MONEY`, without the rupee sign or the word *crore*. The same model therefore labels the same city differently in two sentences. We did not isolate the cause, so it remains uncertain. Chapter 13 returns to such differences.

Sentencizer

A sentencizer determines sentence boundaries with simple rules. It is useful when you need only sentence segmentation and not a full trained parser. A blank pipeline does not contain it by default, so it is added explicitly. Section 7.8 introduced the sentencizer; the example below is the notes' version.

```
blank = spacy.blank("en")
blank.add_pipe("sentencizer")

doc = blank("I am learning NLP. spaCy is useful.")
for sent in doc.sents:
    print(sent.text)
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
I am learning NLP.
spaCy is useful.
```

10.6 Modular Pipelines and Custom Components

One of spaCy's design principles is **modularity**: you do not have to use every available component. An application that only extracts organisations needs the tokenizer and NER and nothing else (Figure 10.2, right). Three applications might need three different pipelines: tokenizer plus NER; tokenizer plus tagger plus lemmatizer; or tokenizer plus parser plus NER. Using only the required components makes an application more lightweight and focused, and can reduce memory use, processing time and complexity.

Adding components with add_pipe()

Components are added with `nlp.add_pipe()`. The general form is `nlp.add_pipe("component_name")`. The tokenizer is handled separately from the component list, so a blank pipeline with a sentencizer reports only one component name:

```
blank = spacy.blank("en")
blank.add_pipe("sentencizer")
print(blank.pipe_names)
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
['sentencizer']
```

Reusing a trained component

A component can also be taken from a trained pipeline. Suppose we need only entities. We create a blank pipeline, load a trained model, and copy just the NER component with `source=`:

```
nlp_custom = spacy.blank("en")
print(nlp_custom.pipe_names)

en_model = spacy.load("en_core_web_sm")
nlp_custom.add_pipe("ner", source=en_model)
print(nlp_custom.pipe_names)

doc = nlp_custom("Apple opened a new office in Chennai.")
for ent in doc.ents:
    print(ent.text, ent.label_)
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
[]
['ner']
Apple ORG
Chennai GPE
```

In `nlp.add_pipe("ner", source=en_model)`, `"ner"` identifies the component and `source=en_model` names the trained pipeline from which it is obtained. Printing `pipe_names` afterwards verifies the configuration. The resulting pipeline contains only the tokenizer and NER, yet it still finds the entities. The other components of `en_core_web_sm` were not copied, so this pipeline does no tagging, parsing or lemmatization.

10.7 Pipeline Order Matters

Components run in a fixed order, and the output of one component can be used by later ones. A spaCy pipeline is therefore not just a collection of components but an ordered processing sequence. If a component depends on an annotation produced elsewhere, the component that produces it must be present and must run first.

The following experiment makes this concrete. We build four small pipelines from components of `en_core_web_sm` and print each token's part of speech and lemma for the same sentence.

```
src = spacy.load("en_core_web_sm")
text = "The students are studying."

def build(names):
    p = spacy.blank("en")
    for n in names:
        p.add_pipe(n, source=src)
    return p

orders = [
    ["lemmatizer"],
    ["tok2vec", "tagger", "lemmatizer"],
    ["tok2vec", "tagger", "attribute_ruler", "lemmatizer"],
```

```
    ["tok2vec", "tagger", "lemmatizer", "attribute_ruler"],
]
for names in orders:
    p = build(names)
    print(names)
    print("  ", [(t.text, t.pos_, t.lemma_) for t in p(text)])
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
['lemmatizer']
   [('The', '', 'the'), ('students', '', 'students'), ('are', '', 'are'),
('studying', '', 'studying'), ('.', '', '.')]
['tok2vec', 'tagger', 'lemmatizer']
   [('The', '', 'the'), ('students', '', 'students'), ('are', '', 'are'),
('studying', '', 'studying'), ('.', '', '.')]
['tok2vec', 'tagger', 'attribute_ruler', 'lemmatizer']
   [('The', 'DET', 'the'), ('students', 'NOUN', 'student'), ('are', 'AUX',
'be'), ('studying', 'VERB', 'study'), ('.', 'PUNCT', '.')]
['tok2vec', 'tagger', 'lemmatizer', 'attribute_ruler']
   [('The', 'DET', 'the'), ('students', 'NOUN', 'students'), ('are', 'AUX',
'be'), ('studying', 'VERB', 'studying'), ('.', 'PUNCT', '.')]
```

Only the third pipeline reproduces the lemmas of the full pipeline (*student*, *be*, *study*). In the first two the `pos_` values are empty and the words come back unchanged. In the fourth the lemmatizer runs before `pos_` has been set, so most words stay unchanged and only *are* becomes *be*. In this model the lemmatizer relies on the `pos_` values, the tagger predicts the detailed tags (`tag_`), and the `attribute_ruler` is the component that fills in `pos_` from them. Without the `attribute_ruler`, or with it placed after the lemmatizer, the lemmatizer has no part-of-speech information to use. Order is part of the design of a pipeline. Section 13.7 shows a second example in which the position of the EntityRuler decides which entity labels are kept.

This experiment is a statement about `en_core_web_sm` 3.8.0. Other models may arrange their components differently.

10.8 Multilingual Pipelines

spaCy supports many languages, and the language code selects the language-specific rules. `spacy.blank("en")` creates an English blank pipeline, `spacy.blank("fr")` a French one and `spacy.blank("hi")` a Hindi one (Section 7.6 shows multilingual tokenization). A blank language pipeline does not mean that a trained model exists for that language: `spacy.blank("fr")` gives a French language object with its tokenizer, and a trained French model, if available, must be installed and loaded separately to obtain tagging or NER.

```
french = spacy.blank("fr")
doc = french("Bonjour, je suis étudiant.")
print([token.text for token in doc])
print(french.pipe_names)
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
['Bonjour', ',', 'je', 'suis', 'étudiant', '.']
[]
```

The second line confirms that the blank French pipeline contains no components. We did not install or run a trained French model for this book, so no output is shown for one.

10.9 Library, Language Object, Pipeline, Component and Model

These terms are often confused. Table 10.3 separates them.

Term	Meaning	Example
spaCy	The NLP framework or library	`import spacy`
Language object	Manages the tokenizer and the pipeline; created by `blank()` or loaded from a model	`spacy.blank("en")`
Tokenizer	Converts raw text into tokens	—
Pipeline	The ordered collection of processing components	`nlp.pipe_names`
Component	One processing unit inside the pipeline	`tagger`, `parser`, `ner`, `lemmatizer`, `sentencizer`
Pre-trained model	A trained set of components and their learned parameters	`en_core_web_sm`

Table 10.3 Library, language object, tokenizer, pipeline, component and model.

10.10 Choosing a Pipeline

If you need...	Use
only tokenization	`nlp = spacy.blank("en")`
ready-made NLP analysis	`nlp = spacy.load("en_core_web_sm")`
sentence segmentation only	`nlp = spacy.blank("en")` followed by `nlp.add_pipe("sentencizer")`
a custom combination of components	`nlp = spacy.blank("en")`, then add the required components

Table 10.4 A practical decision guide.

10.11 Common Mistakes

- **Expecting analysis from a blank pipeline.** `spacy.blank("en")` returns tokens only; `pos_`, `dep_` and `ents` stay empty (Section 10.3).
- **Treating a language code as a trained model.** `spacy.blank("fr")` is not a French model (Section 10.8).
- **Assuming the component list is the same everywhere.** Print `nlp.pipe_names` instead of relying on a remembered list (Section 10.4).
- **Adding components in the wrong order.** A component cannot use an annotation that a later component creates (Section 10.7).
- **Loading more than the application needs.** If only entities are required, consider a custom pipeline (Section 10.6).

- **Confusing installing spaCy with installing a model.** `pip install spacy` does not download `en_core_web_sm`.

10.12 Chapter Summary

- A spaCy pipeline is an ordered sequence of NLP processing components that add information to a `Doc`.
- The tokenizer converts raw text into tokens; the components analyse them. The two are related but not the same.
- A blank pipeline has the language-specific tokenizer and no trained components; a pre-trained pipeline provides ready-made analysis; a custom pipeline contains only the components you select.
- The main components are the tagger, lemmatizer, parser, NER and sentencizer, plus custom components.
- `nlp.pipe_names` and `nlp.pipeline` show what a pipeline contains. In `en_core_web_sm` 3.8.0 the order is tok2vec, tagger, parser, attribute_ruler, lemmatizer, ner.
- `nlp.add_pipe("ner", source=en_model)` reuses a trained component in a custom pipeline.
- Component order matters: the output of one component can be the input of the next.
- spaCy supports many languages, but a blank language pipeline is not a trained model for that language.

10.13 Review Questions and Exercises

1. Explain the difference between the tokenizer and a pipeline component.
2. What does a blank English pipeline contain, and what does it not contain? What is `doc.ents` for a blank pipeline?
3. Why does `pip install spacy` not give you POS tagging?
4. In the experiment of Section 10.7, why are the lemmas unchanged when the `attribute_ruler` is missing?
5. Why does `spacy.blank("fr")` not give you French named entity recognition?
6. **Exercise.** Load `en_core_web_sm`, print `nlp.pipe_names`, and process a sentence of your own. Print the text, `pos_`, `dep_` and `lemma_` of every token and list which component produced each attribute.
7. **Exercise.** Build a custom pipeline that contains only the `tagger` and its required components. Which attributes are filled and which stay empty? Explain the result.
8. **Exercise.** Create a blank pipeline with a sentencizer and compare its sentences with those of a pre-trained pipeline on a text with abbreviations such as “Dr. Rao”.

Chapter 11 Part-of-Speech Tagging

Learning objectives

After studying this chapter you will be able to:

- define part-of-speech (POS) tagging and explain why it is useful;
- name the traditional parts of speech and relate them to spaCy's tag names;
- read `token.pos_` and `token.tag_` and explain how they differ;
- use `spacy.explain()` to interpret tag codes;
- filter tokens by POS and count POS categories with `count_by()`;
- describe typical applications of POS tagging and its limits.

Chapter 10 introduced the tagger as one component of a spaCy pipeline. This chapter looks at what it produces. POS tagging identifies the grammatical role of each word in a sentence, and it is one of the most widely used building blocks in linguistic analysis. It also prepares the ground for Chapter 12, where those roles are connected into a sentence structure.

11.1 What Is POS Tagging?

Part-of-speech (POS) tagging is a technique that identifies the grammatical role of each word in a sentence. Words can function differently depending on how they are used, and POS tagging helps computers understand these roles automatically, using a trained NLP model. For the sentence "The smart student studies carefully." the intended analysis is shown in Table 11.1.

Word	POS tag	Grammatical role
The	`DET`	Determiner
smart	`ADJ`	Adjective
student	`NOUN`	Noun
studies	`VERB`	Verb
carefully	`ADV`	Adverb
.	`PUNCT`	Punctuation

Table 11.1 The intended POS analysis of the example sentence (from the notes).

POS tagging is useful in text preprocessing and cleaning, sentiment analysis, text classification, information extraction, named entity recognition (Chapter 13), machine translation, question-answering systems and grammar analysis.

11.2 Types of Parts of Speech

Traditional English grammar has eight major parts of speech. Table 11.2 lists them with the spaCy tag names, examples and the example sentences used in the notes.

Part of speech	Meaning	Examples	Example sentence
Noun (NOUN)	A person, place, thing or idea	student, Chennai, book, intelligence	The **student** reads a **book**.
Verb (VERB)	An action, occurrence or state of being	run, eat, write, is, think	She **writes** a letter.
Pronoun (PRON)	Replaces or refers to a noun	he, she, it, they, we	**They** are studying.
Adjective (ADJ)	Describes a noun or pronoun	beautiful, red, smart, sweet	The **smart** student answered.
Adverb (ADV)	Modifies a verb, adjective or adverb	quickly, slowly, very, carefully	He runs **quickly**.
Conjunction (CCONJ / SCONJ)	Connects words, phrases or clauses	and, but, or, because, although	Ram **and** Sam are friends.
Preposition (ADP)	Expresses place, time, direction or other relations	in, on, at, under, between	The book is **on** the table.
Interjection (INTJ)	A sudden feeling or emotion	wow, alas, oops, hurray	**Wow!** That is amazing.

Table 11.2 The eight traditional parts of speech and their spaCy tag names.

In the conjunction example, *and* is a coordinating conjunction (CCONJ); subordinating conjunctions such as *because* are tagged SCONJ. spaCy uses a more detailed system than the traditional eight categories. For example, it distinguishes determiners (DET), numerals (NUM), particles (PART) and punctuation (PUNCT) as separate categories, and it has the categories AUX (auxiliary verb), PROPN (proper noun), SPACE and X (other) that appear in the examples below. Appendix C will list the complete tag sets.

11.3 POS Tagging with spaCy

spaCy is an open-source Python library for NLP. It supports tokenization, POS tagging, lemmatization, dependency parsing, named entity recognition and text analysis, and it uses pre-trained language models to assign grammatical labels to tokens. Install the library and download the small English model as shown in Section 10.3, then run the notes' program:

```
import spacy

# Load the English language model
nlp = spacy.load("en_core_web_sm")

# Input sentence
text = "The smart student studies carefully."

# Process the text
doc = nlp(text)

# Display POS tags
for token in doc:
    print(token.text, token.pos_, token.tag_)
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
The DET DT
smart ADJ JJ
student NOUN NN
studies NOUN NNS
carefully ADV RB
. PUNCT .
```

The code does five things. `import spacy` imports the library; `spacy.load()` loads the pre-trained English model; `nlp(text)` processes the sentence and creates a `Doc` with tokens and annotations; `token.text` returns the original word or punctuation; `token.pos_` returns the general grammatical category and `token.tag_` the more detailed tag.

The notes give the following as possible output and add that the exact output depends on the model and sentence context:

Output given in the notes (not executed here)

```
The DET DT
smart ADJ JJ
student NOUN NN
studies VERB VBZ
carefully ADV RB
. PUNCT .
```

The verified output differs in one token: this model tags *studies* as a plural noun (`NOUN`, `NNS`) instead of a verb. The sentence is ambiguous in isolation because *studies* can be a plural noun or a third-person singular verb, and the model had to choose from the surrounding words. We can look at the evidence directly by changing the sentence slightly:

```
for s in ["The smart students study carefully.",
          "He studies NLP every day."]:
    print([(t.text, t.pos_, t.tag_)
           for t in nlp(s) if t.lower_.startswith("stud")])
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
[('students', 'NOUN', 'NNS'), ('study', 'VERB', 'VBP')]
[('studies', 'VERB', 'VBZ')]
```

In the first sentence *study* is a verb (`VBP`), and in the second *studies* is a verb (`VBZ`). The model therefore knows both uses; for the notes' original sentence it chose the noun reading. We did not investigate why, so the cause remains uncertain. The lesson is general: a tagger is a statistical model whose predictions can be wrong, and the result for one model and sentence should not be treated as a universal property of English.

11.4 Coarse and Fine-Grained Tags: pos_ and tag_

spaCy gives two POS attributes (Table 11.3). `token.pos_` is the broad category; `token.tag_` is the detailed tag, and it carries extra grammatical information.

Feature	token.pos_	token.tag_
Meaning	General POS category	Detailed grammatical tag
Detail	Broad	More specific
Example	VERB	VBZ
Information	Identifies a word as a verb	Can identify a third-person singular present-tense verb
Use	General linguistic analysis	Detailed grammatical analysis

Table 11.3 token.pos_ compared with token.tag_.

Consider “She writes code.” For the word *writes*, `token.pos_` returns VERB and `token.tag_` returns VBZ, a present-tense verb in the third-person singular, in the English tagging system used by spaCy’s model (Figure 11.1).

```
doc = nlp("She writes code.")
for token in doc:
    print(token.text, token.pos_, token.tag_)
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
She PRON PRP
writes VERB VBZ
code NOUN NN
. PUNCT .
```

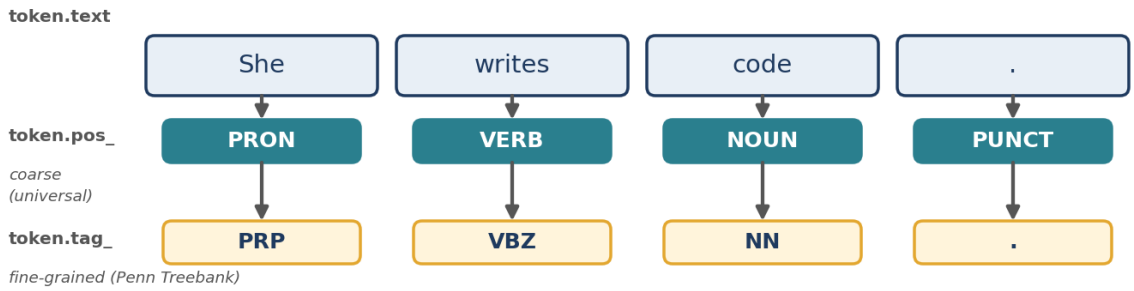


Figure 11.1 Coarse (pos_) and fine-grained (tag_) tags for “She writes code.” (verified output).

Table 11.4 lists other common detailed tags. They are commonly associated with the English Penn Treebank tagging scheme.

Tag	Meaning	Example
NN	Singular noun	student
NNS	Plural noun	students
VB	Base-form verb	write
VBD	Past-tense verb	wrote
VBG	Gerund or present participle	writing
VBN	Past participle	written
VBP	Non-third-person singular present verb	write

Tag	Meaning	Example
VBZ	Third-person singular present verb	writes
JJ	Adjective	beautiful
RB	Adverb	quickly
DT	Determiner	the

Table 11.4 Common detailed tags (examples from the notes).

11.5 Understanding spacy.explain()

The function `spacy.explain()` returns a human-readable explanation of a POS tag or detailed tag. It helps you understand unfamiliar codes, interpret and debug tagging results, and learn by experiment. If you meet the tag `VBD`, `spacy.explain("VBD")` tells you what it means.

```
print(spacy.explain("NOUN"))
print(spacy.explain("VERB"))
print(spacy.explain("VBZ"))
print(spacy.explain("JJ"))
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
noun
verb
verb, 3rd person singular present
adjective (English), other noun-modifier (Chinese)
```

The notes give the possible output *noun*, *verb*, *verb*, *3rd person singular present* and *adjective*. The first three agree with the verified output. For `JJ` the verified explanation also names the Chinese use of the tag, because the explanation table covers more than one language; for English text it is simply an adjective.

11.6 Filtering Tokens by POS

Text often contains punctuation, spaces and other tokens that are not useful for a particular analysis. Filtering by POS keeps only the tokens relevant to the task. When analysing meaningful words we may exclude spaces (`SPACE`), punctuation (`PUNCT`) and other or miscellaneous tokens (`X`).

```
text = "Wow! The students are learning NLP."
doc = nlp(text)

for token in doc:
    if token.pos_ not in ["SPACE", "PUNCT", "X"]:
        print(token.text, token.pos_)
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
Wow INTJ
The DET
students NOUN
are AUX
learning VERB
NLP PROPN
```

The condition `token.pos_ not in ["SPACE", "PUNCT", "X"]` checks whether the token belongs to an excluded category: such tokens are skipped and all others are kept. The verified output equals the notes' possible output (Wow INTJ, The DET, students NOUN, are AUX, learning VERB, NLP PROPN).

Filtering should depend on the application. Punctuation may matter for sentiment analysis, and auxiliary verbs such as *are* may matter for grammatical analysis. Do not remove tokens that the task needs. Compare the stop-word discussion in Section 8.3.

11.7 Counting POS Tags with count_by()

The `count_by()` method counts a linguistic attribute across the tokens of a document. It is useful for long reports, articles and reviews.

```
text = """Artificial intelligence helps students.
Students learn Python and develop applications."""
doc = nlp(text)

# Count tokens by POS category
pos_counts = doc.count_by(spacy.attrs.POS)

for pos_id, count in pos_counts.items():
    print(nlp.vocab[pos_id].text, count)
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
ADJ 1
NOUN 4
VERB 3
PUNCT 2
SPACE 1
PROPN 1
CCONJ 1
```

`doc.count_by(spacy.attrs.POS)` returns a dictionary whose keys are numeric attribute IDs; `nlp.vocab[pos_id].text` converts each ID into a readable label. Note that the line break in the text is itself a token and is counted under `SPACE`. The exact counts depend on the input and on the model's tokenization. Figure 11.2 shows the counts.

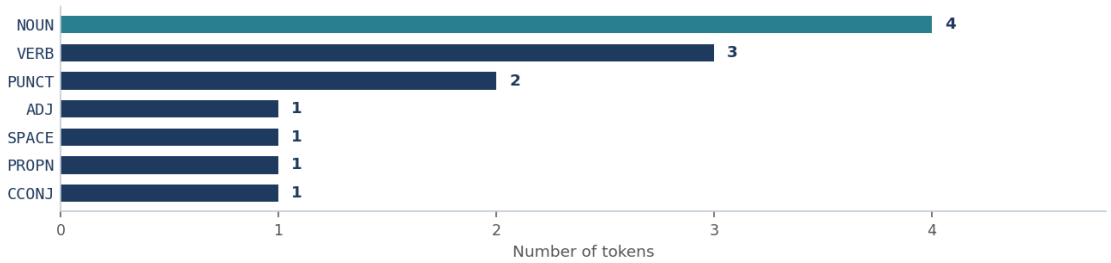


Figure 11.2 POS counts for the two-sentence text above (verified output).

POS frequencies can show how many nouns or verbs a document uses, whether it contains many adjectives, how writing styles differ across documents, and which grammatical categories dominate

a collection. A document with many adjectives and adverbs may use descriptive language, while a technical report may contain many nouns and specialised terms. Such patterns are clues, not proofs.

11.8 Practical Applications of POS Tagging

Application	How POS tags help
Text preprocessing	Identify grammatical categories and select relevant tokens
Sentiment analysis	Identify adjectives and adverbs that may express opinions, such as *excellent*, *terrible* and *slowly*
Information extraction	Identify patterns involving nouns, verbs and other categories
Named entity recognition	Work alongside other components to analyse names, organisations and places (Chapter 13)
Machine translation	Provide grammatical information that can support translation systems
Text analytics	Measure the distribution of grammatical categories in large documents
Question answering	Help understand sentence structure and identify relevant words

Table 11.5 Applications of POS tagging.

11.9 Context Decides the Tag

The notes begin by observing that words can function differently depending on how they are used. The same spelling can therefore receive different tags in different sentences:

```
for s in ["She will book a flight.", "She read a book."]:
    print([(t.text, t.pos_) for t in nlp(s) if t.text.lower() == "book"])
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
[('book', 'VERB')]
[('book', 'NOUN')]
```

Here *book* is a verb in the first sentence and a noun in the second. This is why a tagger must look at the context rather than at the word alone, and why a dictionary lookup of single words cannot replace it.

11.10 Common Mistakes

- **Treating the output as ground truth.** The model tagged *studies* as a noun in the notes' own example sentence (Section 11.3). Always inspect samples of the output.
- **Confusing `pos_` with `tag_`.** `pos_` is the broad category and `tag_` the fine-grained tag (Section 11.4).
- **Forgetting the underscore.** `token.pos` returns an integer ID; `token.pos_` returns the readable label.
- **Using a blank pipeline.** Without a tagger and an `attribute_ruler`, `pos_` is empty (Section 10.7).
- **Removing tokens by POS without thinking.** Punctuation and auxiliary verbs can carry meaning (Section 11.6).

- **Over-interpreting POS counts.** They hint at style; they do not prove it (Section 11.7).

11.11 Chapter Summary

- POS tagging assigns a grammatical label to each token; it supports preprocessing, extraction, classification and analysis.
- English has eight traditional parts of speech; spaCy's tag set is more detailed and includes `DET`, `NUM`, `PART`, `PUNCT`, `AUX`, `PROPN`, `SPACE` and `X`.
- `token.pos_` is the broad category and `token.tag_` the fine-grained tag; `spacy.explain()` explains the codes.
- Tokens can be filtered by POS, and `doc.count_by(spacy.attrs.POS)` counts categories.
- Tagging is context-dependent and statistical, so errors occur; check the output of the model you use.

11.12 Review Questions and Exercises

1. Name the eight traditional parts of speech and give one example sentence for each.
2. What is the difference between `token.pos_` and `token.tag_`? Give an example for the word *writes*.
3. What does `spacy.explain("VBD")` return, and why is the function useful?
4. Why can the same word receive different tags in different sentences?
5. **Exercise.** Tag a paragraph of your own and list the tokens whose `pos_` is `NOUN`, `PROPN` or `VERB`. Check ten of them by hand and report any errors.
6. **Exercise.** Use `count_by()` to compare the POS distribution of a product review and a news paragraph. What differences do you see, and how cautious should you be about them?
7. **Exercise.** Write a function that returns only the adjectives and adverbs of a sentence, and use it on three reviews. Explain one case in which it gives a misleading result.

Chapter 12 Dependency Parsing

Learning objectives

After studying this chapter you will be able to:

- explain what a dependency parse is, using the terms head, dependent, relation and root;
- read `token.dep_` and `token.head` for every token of a sentence;
- interpret common dependency labels with `spacy.explain()`;
- navigate the tree with `children`, `subtree` and `ancestors`;
- render a parse with displaCy;
- extract simple subject–verb–object relations, and use parser-based sentence boundaries;
- recognise typical situations in which the parser is wrong.

Part-of-speech tags (Chapter 11) say what each word is. They do not say how the words relate to one another. The dependency parser adds that information: it connects the words of a sentence into a structure. This chapter explains the structure and then uses it for relation extraction and sentence segmentation.

12.1 What Is Dependency Parsing?

The **dependency parser** analyses the grammatical relationships between the words of a sentence. In "Mani reads a book." the parser can identify that *Mani* is the subject of *reads* and that *book* is the object of *reads*. This lets spaCy represent the grammatical structure of the sentence. Dependency parsing is useful for understanding sentence structure, extracting relationships, question answering, information extraction and linguistic analysis.

A dependency parse links words in pairs. In each pair one word is the **head** and the other is its **dependent**, and the link carries a **relation** label such as "subject" or "object". Exactly one word in each sentence, usually the main verb, is the **root**; it is the head of the whole sentence and has no head of its own (spaCy lists it as its own head).

12.2 Dependency Parsing with spaCy

Every token has two dependency attributes: `token.dep_` gives the relation to its head and `token.head` gives the head token itself. The notes' example prints both for each token.

```
import spacy

nlp = spacy.load("en_core_web_sm")
doc = nlp("Mani reads a book.")

for token in doc:
    print(token.text, token.dep_, token.head.text)
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
Mani nsubj reads
reads ROOT reads
```

```
a det book
book dobj reads
. punct reads
```

The notes describe the result conceptually as three relationships:

Output given in the notes (not executed here)

```
Mani   → subject → reads
reads  → root    → reads
book   → object  → reads
```

The verified output agrees with this description and adds two more tokens: the determiner *a*, attached to *book*, and the full stop, attached to the root. spaCy's English model names the relations `nsubj` (subject), `ROOT` and `dobj` (direct object). Note that *reads* is its own head: that is how spaCy marks the root.

12.3 Dependency Labels

Table 12.1 lists the labels that occur in this chapter's examples. The explanations are produced by `spacy.explain()`, which works for dependency labels as it does for POS tags (Section 11.5).

Label	Explanation (spacy.explain)	Example
`nsubj`	nominal subject	Apple (subject of *opening*)
`aux`	auxiliary	*is*
`ROOT`	root	*opening*
`dobj`	direct object	*office*
`det`	determiner	*a*
`amod`	adjectival modifier	*new*
`prep`	prepositional modifier	*in*
`pobj`	object of preposition	*Chennai*
`punct`	punctuation	the full stop
`nsubjpass`	nominal subject (passive)	*office* in "The office was opened by Apple."
`agent`	agent	*by* in the same sentence

Table 12.1 Dependency labels used in this chapter. The label set belongs to the English model and differs between models and languages. Appendix C will list the complete set.

12.4 Head, Children and Subtree

Because every token has a head, a sentence forms a tree. The root is at the top, and `token.children` lists the tokens that depend directly on a token. `token.subtree` lists the token itself and everything below it, so it gives the complete phrase headed by that token; `token.ancestors` goes the other way, up through the heads. Figure 12.1 draws the parse of the sentence from Chapter 10 and the code reads the same structure.

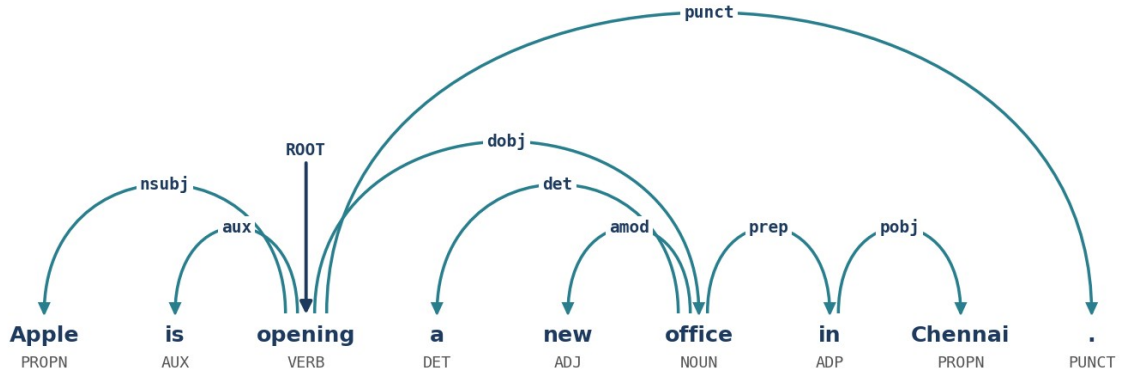


Figure 12.1 Dependency parse of "Apple is opening a new office in Chennai." Each arc runs from a head to its dependent; ROOT marks the main verb. Redrawn from the verified output below.

```
doc = nlp("Apple is opening a new office in Chennai.")

for token in doc:
    print(token.i, token.text, token.dep_, token.head.text,
          [child.text for child in token.children])
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
0 Apple nsubj opening []
1 is aux opening []
2 opening ROOT opening ['Apple', 'is', 'office', '.']
3 a det office []
4 new amod office []
5 office dobj opening ['a', 'new', 'in']
6 in prep office ['Chennai']
7 Chennai pobj in []
8 . punct opening []
```

Reading the table from the root: *opening* has four children (*Apple*, *is*, *office* and the full stop), and *office* has three (*a*, *new* and *in*). The prepositional phrase hangs below *office* and *Chennai* below *in*. Subtrees return whole phrases:

```
office = doc[5]
print([t.text for t in office.subtree])
print([t.text for t in doc[7].ancestors])
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
['a', 'new', 'office', 'in', 'Chennai']
['in', 'office', 'opening']
```

The subtree of *office* is "a new office in Chennai", a complete noun phrase, and the ancestors of *Chennai* are *in*, *office* and *opening*. The subtree of the root is the whole sentence.

12.5 Visualizing a Parse with displaCy

Section 13.5 introduces displaCy for entities. The same tool draws dependency parses when the style is `"dep"`. In a notebook use `displacy.render(doc, style="dep", jupyter=True)`; in a script `displacy.serve(doc, style="dep")` starts a small web server. Both draw the tokens with arcs between heads and dependents, as in Figure 12.1. `displacy.render()` can also return the markup as a string:

```
from spacy import displacy

doc = nlp("Mani reads a book.")
svg = displacy.render(doc, style="dep", jupyter=False)
print(type(svg).__name__, svg[:4])
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
str <svg
```

12.6 Extracting Subject–Verb–Object Relations

The notes list relationship extraction and information extraction as uses of the parser. The simplest relation is a triple of subject, verb and object, which we can read off the tree: find a verb that has a child labelled `nsubj` and a child labelled `dobj`.

```
def extract_svo(doc):
    triples = []
    for token in doc:
        if token.pos_ == "VERB":
            kids = list(token.children)
            subjects = [c for c in kids if c.dep_ in ("nsubj", "nsubjpass")]
            objects = [c for c in kids if c.dep_ == "dobj"]
            if subjects and objects:
                triples.append((subjects[0].text, token.lemma_,
                                objects[0].text))
    return triples

for s in ["Mani reads a book.",
          "Apple is opening a new office in Chennai.",
          "The office was opened by Apple.",
          "He was an Indian aerospace scientist."]:
    print(s, extract_svo(nlp(s)))
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
Mani reads a book. [('Mani', 'read', 'book')]
Apple is opening a new office in Chennai. [('Apple', 'open', 'office')]
The office was opened by Apple. []
He was an Indian aerospace scientist. []
```

The function returns the lemma of the verb (Chapter 9), so *reads* and *opening* become *read* and *open*. The last two sentences return nothing. In the passive sentence the grammatical subject is *office* (`nsubjpass`) and the doer *Apple* sits inside an `agent` phrase; in the last sentence the verb

was links to its complement with the label `attr`, and there is no `dobj`. A usable extractor must handle these patterns too. This function is a small illustration, not a complete relation extractor.

12.7 Parser-Based Sentence Segmentation

Section 7.8 explained that a pre-trained pipeline finds sentence boundaries with the parser, whereas a blank pipeline needs the rule-based sentencizer. The parser sets a sentence start from the structure of the sentence, so every sentence has exactly one root. With the biography from Chapter 5 (Section 5.6):

```
bio = ("A. P. J. Abdul Kalam was born on 15 October 1931 in Rameswaram, "
       "Tamil Nadu. He was an Indian aerospace scientist and served as "
       "the 11th President of India.")
doc = nlp(bio)
for sent in doc.sents:
    print(sent.text)
    print("   root:", sent.root.text)
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
A. P. J. Abdul Kalam was born on 15 October 1931 in Rameswaram, Tamil Nadu.
   root: born
He was an Indian aerospace scientist and served as the 11th President of
India.
   root: was
```

Abbreviations such as the initials of “A. P. J.” do not cause false splits, and each sentence reports its own root verb. The parser can also place boundaries where there is no punctuation to guide a rule. We compare it with the sentencizer on a run-on text:

```
sentencizer = spacy.blank("en")
sentencizer.add_pipe("sentencizer")

text = "i went home and then i slept it was late"
print("parser     :", [s.text for s in nlp(text).sents])
print("sentencizer:", [s.text for s in sentencizer(text).sents])
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
parser     : ['i went home', 'and then i slept it was late']
sentencizer: ['i went home and then i slept it was late']
```

The sentencizer finds no punctuation and returns one sentence. The parser returns two, but its boundary falls before *and then i slept*, not before *it was late*, which is where most readers would split. Parser-based segmentation is therefore more flexible than a punctuation rule, but it is not perfect. As always, results come from this model and these examples only.

12.8 When the Parse Is Wrong

A statistical parser makes mistakes, especially where the structure of a sentence is genuinely ambiguous. Compare the following two sentences, which have the same shape but need different attachments of the phrase beginning with *with*:

```
for s in ["She ate pizza with a fork.", "She ate pizza with mushrooms."]:
    doc = nlp(s)
    print(s, [(t.text, t.head.text) for t in doc if t.dep_ == "prep"])
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
She ate pizza with a fork. [('with', 'pizza')]
She ate pizza with mushrooms. [('with', 'pizza')]
```

In both sentences the parser attaches *with* to *pizza*. That is plausible for the mushrooms but not for the fork, which is the instrument of the eating. Choosing the right attachment requires knowledge of the world, not just the words. Chapters 11 to 13 each show a pre-trained model making plausible but wrong decisions, so inspect sample output before building on it, and evaluate the system (Chapter 22).

12.9 Common Mistakes

- **Forgetting that the root is its own head.** `token.head` of the root returns the root itself (Section 12.2).
- **Using the tag set of one model for another.** Labels such as `dobj` belong to the English model; other models and languages use different sets (Table 12.1).
- **Reading only `token.dep_`.** The label needs `token.head` to be meaningful.
- **Assuming every verb has a subject and an object.** Passive and copular sentences return no triple (Section 12.6).
- **Relying on the parse of unpunctuated text.** Boundaries can be misplaced (Section 12.7).
- **Trusting every attachment.** Prepositional phrases are a frequent source of errors (Section 12.8).

12.10 Chapter Summary

- The dependency parser links each word to a head with a relation label; the main verb is the root.
- `token.dep_` is the relation and `token.head` the head; `children`, `subtree` and `ancestors` navigate the tree.
- `spacy.explain()` explains dependency labels; the label set belongs to the model.
- displaCy with `style="dep"` draws a parse.
- Subject–verb–object triples can be read from `nsubj`, the verb and `dobj`, but passive and copular sentences need extra rules.
- Pre-trained pipelines use the parser for sentence boundaries; it can work without punctuation but is not always right.
- The parser is a statistical model, so wrong attachments occur.

12.11 Review Questions and Exercises

1. Define head, dependent, relation and root. Which token is the root of “Mani reads a book.”?
2. What do `token.children` and `token.subtree` return? Why does the subtree of the root contain the entire sentence?

3. Why does `extract_svo` return nothing for “The office was opened by Apple.”?
4. How does the way a pre-trained pipeline finds sentence boundaries differ from the sentencizer?
5. **Exercise.** Parse three sentences of your own and draw each tree by hand from the `token.head` values. Compare your drawings with displaCy.
6. **Exercise.** Extend `extract_svo` so that it also reports passive sentences as (agent, verb, subject). Test it on three sentences and report where it fails.
7. **Exercise.** Find two sentences of your own in which the parser attaches a prepositional phrase wrongly. Explain what knowledge the correct attachment needs.

Chapter 13 Named Entity Recognition

Learning objectives

After studying this chapter you will be able to:

- define named entity recognition (NER) and name the common entity labels;
- extract entities with `doc.ents` and visualise them with displaCy;
- add or correct entities with `Span` and `doc.set_ents()`;
- compare lookup-based, rule-based and machine-learning approaches, and use the EntityRuler;
- explain why the position of the EntityRuler in the pipeline matters;
- describe how NER is evaluated, and read pre-trained results critically.

Chapter 5 extracted a person's name, date of birth and birthplace from a short biography with regular expressions, and noted that the patterns break when the wording changes. Named entity recognition is the trained component that finds such items without hand-written patterns. This chapter closes Part IV and returns to that biography in Section 13.10.

13.1 What Is Named Entity Recognition?

Named entity recognition (NER) is an NLP technique that identifies and classifies named entities in a text into predefined categories. Named entities are real-world objects such as people, organisations, locations, dates, monetary values and products. NER lets a computer find out who, where, when and what a text is about. For the notes' example sentence "Elon Musk founded SpaceX in the United States in 2002." the intended analysis is shown in Table 13.1.

Entity	Entity label	Meaning
Elon Musk	`PERSON`	Name of a person
SpaceX	`ORG`	Organization
the United States	`GPE`	Geopolitical entity
2002	`DATE`	Date or year

Table 13.1 The intended entity analysis of the example sentence (from the notes).

Section 13.4 shows what a trained pipeline actually returns for this sentence. It does not agree with Table 13.1 in every row.

13.2 Types of Named Entities

Table 13.2 lists the common entity categories used in spaCy's English models. These are standard labels, although the actual recognition depends on the model and the context.

Entity label	Description	Example
`PERSON`	People	Elon Musk
`ORG`	Organizations	Google, Microsoft

Entity label	Description	Example
GPE	Countries, cities and states	India, Chennai
LOC	Geographical locations	Himalayas
DATE	Dates or periods	October 18, 2026
TIME	Times of day	10:30 AM
MONEY	Monetary values	₹5,000, $100
PERCENT	Percentages	25%, 50%
PRODUCT	Products	iPhone
EVENT	Named events	Olympics
FAC	Buildings and facilities	Eiffel Tower
NORP	Nationalities, religious or political groups	Indians, Republicans

Table 13.2 Common entity labels (from the notes).

NER identifies the entity and its category, not merely the grammatical role of a word. The notes illustrate this with "Microsoft hired Ravi in Chennai with a salary of ₹50,000." and expect Microsoft → ORG, Ravi → PERSON, Chennai → GPE and ₹50,000 → MONEY. The verified output for this sentence is:

```
import spacy

nlp = spacy.load("en_core_web_sm")
doc = nlp("Microsoft hired Ravi in Chennai with a salary of ₹50,000.")
print([(ent.text, ent.label_) for ent in doc.ents])
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
[('Microsoft', 'ORG'), ('Ravi', 'ORG'), ('Chennai', 'GPE'), ('50,000',
'MONEY')]
```

Two expectations are met exactly (Microsoft and Chennai). The model labels *Ravi* as an organisation, not a person, and the amount is returned with the correct label but as `50,000`, without the rupee sign. The cause is not established here. A small model, an unfamiliar name and the sentence context could all play a part, but we did not test them separately, so it remains uncertain.

To see how far Table 13.2 can be relied on, we tested one short sentence for each label with the same model:

Expected label	Test sentence	Entities returned
PERSON	Elon Musk met Sundar Pichai.	Sundar Pichai (PERSON)
ORG	Google and Microsoft grew.	Google (ORG), Microsoft (ORG)
GPE	She moved from India to Chennai.	India (GPE), Chennai (ORG)
LOC	They climbed the Himalayas.	no entity found
DATE	The meeting is on October 18, 2026.	October 18, 2026 (DATE)
TIME	The call is at 10:30 AM.	10:30 AM (TIME)

Expected label	Test sentence	Entities returned
MONEY	It costs ₹5,000 or $100.	₹5,000 or $100 (MONEY)
PERCENT	Sales grew 25%, then 50%.	25% (PERCENT), 50% (PERCENT)
PRODUCT	He bought an iPhone.	iPhone (ORG)
EVENT	The Olympics start soon.	Olympics (EVENT)
FAC	We visited the Eiffel Tower.	the Eiffel Tower (LOC)
NORP	Indians and Republicans voted.	Indians (NORP), Republicans (NORP)

Table 13.3 What en_core_web_sm 3.8.0 returned for one test sentence per label (verified). One sentence per label is an illustration, not an evaluation.

The results mix agreement and disagreement with Table 13.2. They show that the label in a table of examples is what the scheme intends, not a guarantee of what a particular model will output for a particular sentence. Section 13.9 explains how to measure this properly.

13.3 Why Is NER Useful?

Application	How NER helps
Search engines	Entity information helps a search engine understand the people, companies and places in documents. An article that mentions Microsoft and OpenAI can be associated with both organisations.
Recommendation systems	Entities such as people, organisations, products and events can be identified in content; a user who watches films with a particular actor can be shown related films.
Customer support	Important details can be extracted from messages and requests routed to the right department. In “I cannot access my Python Programming course.” the course name can be found if a suitable custom model or rule is configured.
Information extraction	Useful information is extracted from unstructured reports, e-mails, news articles and documents.
Healthcare	Diseases, medicines and symptoms can be identified when a healthcare-specific model is used.
Financial analysis	Company names, currencies, monetary amounts and dates can be identified in financial documents.

Table 13.4 Applications of NER.

13.4 Implementing NER with spaCy

Install spaCy and the English model as in Section 10.3. The program below is the notes’ example. The entities are read from `doc.ents`; each has a text (`ent.text`) and a category (`ent.label_`). The standard attribute is `doc.ents`, not `doc.entities`.

```
import spacy

# Load the pretrained English model
nlp = spacy.load("en_core_web_sm")
```

```
# Input text
text = """Elon Musk founded SpaceX.
The company operates in the United States."""

# Process the text
doc = nlp(text)

# Extract named entities
for ent in doc.ents:
    print(ent.text, "->", ent.label_)
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
Elon Musk -> PERSON
the United States -> GPE
```

The notes give this as possible output:

Output given in the notes (not executed here)

```
Elon Musk -> PERSON
SpaceX -> ORG
the United States -> GPE
```

The verified output contains the other two entities, *Elon Musk* and *the United States*, but *SpaceX* is not found. The same happens in the longer sentence of Section 13.1, so the miss is reproducible for this model. We did not test other models or sizes. Whether a larger English model recognises *SpaceX* is therefore not something we can claim, and the reason for the miss remains uncertain. The practical consequence is the subject of the rest of the chapter: when a pre-trained model misses or mislabels entities that matter to you, you can correct them (Section 13.6), add rules (Section 13.7) or train a model (Section 13.7).

13.5 Visualizing Entities with displaCy

displaCy is spaCy's built-in visualisation tool. With `style="ent"` it highlights named entities and shows their labels, which makes it easy to inspect whether a model identified entities correctly.

```
import spacy
from spacy import displacy

nlp = spacy.load("en_core_web_sm")
text = "Google hired Priya in Bengaluru."
doc = nlp(text)

# Display highlighted entities
displacy.serve(doc, style="ent")
```

When this script runs locally, displaCy starts a small web server and prints a URL in the terminal; open the URL in a browser to see the visualisation and stop the server when finished. In a notebook use `displacy.render(doc, style="ent", jupyter=True)` instead. The server and browser view are not reproducible in a book, so we verify the entities in code and show the result in Figure 13.1.

```
doc = nlp("Google hired Priya in Bengaluru.")
print([(ent.text, ent.label_) for ent in doc.ents])
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
[('Google', 'ORG'), ('Bengaluru', 'GPE')]
```



Figure 13.1 The entities found in "Google hired Priya in Bengaluru.", drawn in the style of displaCy (verified output). The model finds the organisation and the city but not the person.

13.6 Custom Entities with Span and set_ents()

Pre-trained NER models recognise many common entity types but may fail on specialised terms. A college management system, for example, may need course names, department names, student IDs or internal project names. Custom entity annotations let developers add domain-specific information or correct individual predictions.

A `Span` is a continuous range of tokens in a document (Section 7.3). The method `doc.set_ents()` assigns entity annotations to spans. The notes' example labels the phrase "Artificial Intelligence" as a field of study:

```
from spacy.tokens import Span

text = "I am studying Artificial Intelligence."
doc = nlp(text)
print("before:", [(ent.text, ent.label_) for ent in doc.ents])

# Find the exact token range
start = None
end = None
for i, token in enumerate(doc):
    if token.text == "Artificial":
        start = i
    if token.text == "Intelligence":
        end = i + 1

if start is not None and end is not None:
    custom_entity = Span(doc, start, end, label="FIELD_OF_STUDY")
    doc.set_ents([custom_entity], default="unmodified")

for ent in doc.ents:
    print(ent.text, "->", ent.label_)
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
before: [('Artificial Intelligence', 'PERSON')]
Artificial Intelligence -> FIELD_OF_STUDY
```

The notes' expected result, `Artificial Intelligence -> FIELD_OF_STUDY`, is confirmed. The first line adds information the notes do not mention: the pre-trained model had already labelled the same phrase as a person. The argument `default="unmodified"` keeps existing annotations outside the updated span; in the notes' words, an existing entity that overlaps the custom span may need different handling. In this run the overlapping entity was simply replaced by the custom one. We tested only this case.

The same method can correct a wrong prediction, such as the label of *Ravi* in Section 13.2:

```
doc = nlp("Microsoft hired Ravi in Chennai.")
print("before:", [(e.text, e.label_) for e in doc.ents])

i = [t.i for t in doc if t.text == "Ravi"][0]
doc.set_ents([Span(doc, i, i + 1, label="PERSON")], default="unmodified")
print("after: ", [(e.text, e.label_) for e in doc.ents])
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
before: [('Microsoft', 'ORG'), ('Ravi', 'ORG'), ('Chennai', 'GPE')]
after:  [('Microsoft', 'ORG'), ('Ravi', 'PERSON'), ('Chennai', 'GPE')]
```

Manual labelling is useful for small corrections and demonstrations. For large datasets, rule-based methods or a trained NER model are usually more practical.

13.7 Three Approaches to Building an NER System

There are three common approaches to building a custom NER system: a simple lookup, rules, and machine learning.

1. Lookup-based approach

A lookup-based system keeps a predefined list or dictionary of known entities.

```
entities = {
    "Google": "ORG",
    "Microsoft": "ORG",
    "Chennai": "GPE",
    "Python Programming": "COURSE"
}

text = "I am studying Python Programming in Chennai."

for entity, label in entities.items():
    if entity in text:
        print(entity, "->", label)
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
Chennai -> GPE
Python Programming -> COURSE
```

The output equals the notes' output. The approach is simple to implement, fast for known entities and useful when the list is limited and controlled. Its disadvantages are that it cannot reliably

identify unknown entities, may miss variations in spelling or capitalisation, and can produce false matches with plain substring matching. Both problems are easy to demonstrate:

```
text = "I am studying python programming in chennai."
print([e for e in entities if e in text])

places = {"Goa": "GPE"}
text = "Goalkeepers train hard."
print([(e, places[e]) for e in places if e in text])
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
[]
[('Goa', 'GPE')]
```

The first test finds nothing because the capitalisation differs; the second reports the Indian state *Goa* inside the word *Goalkeepers*.

2. Rule-based approach with the EntityRuler

A rule-based system identifies entities with predefined patterns, keywords or regular expressions (Chapter 5). spaCy provides the `EntityRuler` component, which matches patterns and assigns entity labels. The notes add it before the `ner` component:

```
nlp = spacy.load("en_core_web_sm")

ruler = nlp.add_pipe(
    "entity_ruler",
    before="ner"
)

patterns = [
    {"label": "COURSE", "pattern": "Python Programming"},
    {"label": "COURSE", "pattern": "Artificial Intelligence"}
]
ruler.add_patterns(patterns)

doc = nlp("I am learning Python Programming and Artificial Intelligence.")
for ent in doc.ents:
    print(ent.text, "->", ent.label_)
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
Python Programming -> COURSE
Artificial Intelligence -> COURSE
```

This matches the notes' possible output. For comparison, the unmodified pre-trained pipeline gives the following on the same sentence:

```
base = spacy.load("en_core_web_sm")
text = "I am learning Python Programming and Artificial Intelligence."
doc = base(text)
print([(e.text, e.label_) for e in doc.ents])
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
[('Python Programming', 'ORG')]
```

Without the rules the model labels *Python Programming* as an organisation and finds nothing for the second phrase. Patterns given as strings are matched exactly, including capitalisation, so “python programming” in lower case would not be matched by these two patterns.

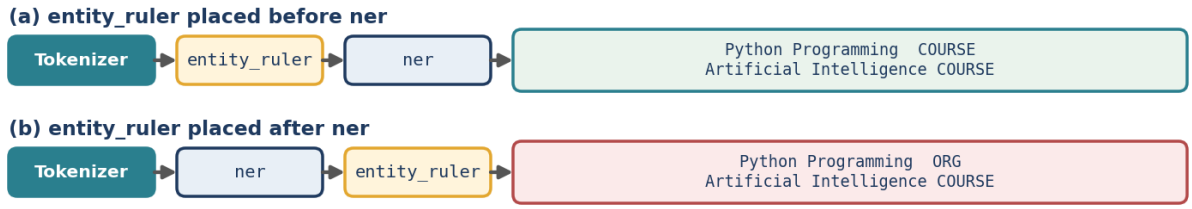
The position of the ruler matters. The notes place it `before="ner"`. If it is placed after `ner`, the entities that `ner` has already found are kept, as the following run shows (compare Section 10.7 and Figure 13.2):

```
late = spacy.load("en_core_web_sm")
late_ruler = late.add_pipe("entity_ruler", after="ner")
late_ruler.add_patterns(patterns)

print(late.pipe_names)
text = "I am learning Python Programming and Artificial Intelligence."
doc = late(text)
print([(e.text, e.label_) for e in doc.ents])
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
['tok2vec', 'tagger', 'parser', 'attribute_ruler', 'lemmatizer', 'ner',
'entity_ruler']
[('Python Programming', 'ORG'), ('Artificial Intelligence', 'COURSE')]
```



Text: "I am learning Python Programming and Artificial Intelligence."
In (a) the rule fires first and its entities are kept; in (b) ner has already labelled "Python Programming".

Figure 13.2 The same patterns with the EntityRuler before and after ner (verified output). Placed before, the rule’s labels are kept; placed after, the label already given by ner remains.

Advantages of rules: they are easy to customise, need no newly trained model and work well for fixed formats and domain-specific terms. Disadvantages: the rules must be designed by hand, may fail when the text differs from the expected patterns, and complex rule sets become hard to maintain.

3. Machine-learning approach

A machine-learning NER system learns to identify entities from annotated training examples instead of depending only on written rules. Common approaches are **conditional random fields (CRF)**, a statistical sequence-labelling method that models relationships between labels in a sequence; **neural networks**, which learn patterns from annotated text; and **transformer models such as BERT**, which use contextual language representations. The development of such a system follows these steps:

- collect text data;
- annotate entity spans and labels;

- divide the data into training, validation and test sets;
- train the model on the annotated examples;
- evaluate its predictions;
- deploy the model and update it when necessary.

Such models can recognise entities that no dictionary lists, learn complex patterns from context and generalise to new examples when the training data is representative. They require annotated data, training and evaluation can be computationally expensive, and performance depends on data quality and on how close the domain is to the training data. Chapter 21 returns to transformer models; training an NER model is outside the scope of this book.

13.8 Comparing the Approaches

Feature	Lookup-based	Rule-based	Machine learning
Main technique	Dictionary matching	Predefined patterns	Learning from data
Training data	Not required	Not required	Usually required
Implementation	Simple	Moderate	More complex
Unknown entities	Limited	Depends on rules	Can generalise
Context understanding	Very limited	Rule-dependent	Often stronger
Best use case	Fixed entity lists	Structured or domain-specific text	Large, varied datasets

Table 13.5 Lookup-based, rule-based and machine-learning NER compared.

In practice NER systems often combine approaches. A pre-trained model can be supplemented with an EntityRuler to recognise organisation names or product codes that the model frequently misses. The two misses from this chapter are an example:

```
combo = spacy.load("en_core_web_sm")
r = combo.add_pipe("entity_ruler", before="ner")
r.add_patterns([{"label": "ORG", "pattern": "SpaceX"},
                {"label": "PERSON", "pattern": "Ravi"}])

for s in ["Elon Musk founded SpaceX in the United States in 2002.",
          "Microsoft hired Ravi in Chennai."]:
    print([(e.text, e.label_) for e in combo(s).ents])
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
[('Elon Musk', 'PERSON'), ('SpaceX', 'ORG'), ('the United States', 'GPE'),
('2002', 'DATE')]
[('Microsoft', 'ORG'), ('Ravi', 'PERSON'), ('Chennai', 'GPE')]
```

With two rules placed before `ner`, *SpaceX* is found and *Ravi* is labelled as a person, while the rest of the pre-trained model's output is kept. Hand-written rules fix only the cases you anticipate.

13.9 Evaluating NER

NER performance is commonly measured with three metrics, which Section 4.2.6 and Table 4.2 introduced for classification in general:

- **Precision:** of all entities predicted by the model, how many were correct?
- **Recall:** of all actual entities, how many did the model identify?
- **F1-score:** the harmonic mean of precision and recall.

F1 = 2 × (Precision × Recall) ÷ (Precision + Recall)

A high F1-score indicates a good balance between precision and recall. For example, high precision with low recall means that the system's predictions are usually correct but that it misses many actual entities. The behaviour of the rule-based and lookup approaches is often of this kind: what they predict is mostly right, but they find only what they were told about. Chapter 22 treats evaluation in general. The small tests in this chapter (Tables 13.2 and 13.3, Section 13.4) are not an evaluation; an evaluation needs a larger set of texts annotated with the correct entities.

13.10 NER on the Chapter 5 Biography

Section 5.6 extracted a name, a birth date, a birthplace and a profession from Abdul Kalam's biography with regular expressions. Here is what the pre-trained pipeline finds in the same text, without any patterns:

```
bio = ("A. P. J. Abdul Kalam was born on 15 October 1931 in Rameswaram, "
       "Tamil Nadu. He was an Indian aerospace scientist and served as "
       "the 11th President of India.")
nlp = spacy.load("en_core_web_sm")
for ent in nlp(bio).ents:
    print(ent.text, "->", ent.label_)
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
A. P. J. Abdul Kalam -> PERSON
15 October 1931 -> DATE
Rameswaram -> GPE
Tamil Nadu -> PERSON
Indian -> NORP
11th -> ORDINAL
India -> GPE
```

The model finds the name (PERSON), the birth date (DATE) and the birthplace (GPE) and also items the regular expressions did not target: a nationality (NORP), an ordinal number (ORDINAL, a label not in Table 13.2) and a country. It also labels *Tamil Nadu* as a person, an error. Next, the same facts in different wording, which Chapter 5 showed breaking the regular expressions:

```
alt = ("Born in Rameswaram in 1931, Abdul Kalam "
       "later became President of India.")
for ent in nlp(alt).ents:
    print(ent.text, "->", ent.label_)
```

Verified output (spaCy 3.8.16, en_core_web_sm 3.8.0)

```
1931 -> DATE
Abdul Kalam -> PERSON
India -> GPE
```

The name, the year and the country are still found, but *Rameswaram* is missed. NER is therefore more robust against rewording than a fixed pattern, but it is not complete or error-free. In practice regular expressions (Chapter 5), trained models and custom rules complement one another.

13.11 Common Mistakes

- **Writing `doc.entities`.** The attribute is `doc.ents` (Section 13.4).
- **Treating model output as ground truth.** The pre-trained model missed *SpaceX*, labelled *Ravi* an organisation and *Tamil Nadu* a person (Sections 13.2, 13.4 and 13.10).
- **Assuming the label table describes every output.** Table 13.2 lists the scheme; Table 13.3 shows what one model returned.
- **Using substring lookups on raw text.** They give false matches and miss capitalisation variants (Section 13.7).
- **Placing the EntityRuler in the wrong position.** It must come before `ner` to take priority (Section 13.7).
- **Skipping evaluation.** A few sentences cannot tell you the quality of an NER system (Section 13.9).

13.12 Chapter Summary

- NER identifies and classifies named entities such as persons, organisations, places, dates and money.
- Entities are read from `doc.ents`; `ent.text` is the string and `ent.label_` the category.
- The labels in a table describe the scheme; the entities a particular model returns can differ, including misses and wrong labels.
- displaCy with `style="ent"` highlights entities for inspection.
- `Span` and `doc.set_ents()` add or correct individual entities.
- Lookup, rule-based (EntityRuler) and machine-learning approaches differ in effort and generalisation; they are often combined.
- The position of the EntityRuler relative to `ner` determines which labels survive.
- NER is evaluated with precision, recall and F1.

13.13 Review Questions and Exercises

1. Define named entity recognition and list five entity labels with an example for each.
2. Why is `doc.entities` wrong? What is the correct attribute?
3. Give two disadvantages of lookup-based NER and one case that demonstrates each.
4. Why can a pre-trained model and an EntityRuler give different labels for the same phrase, and what decides which one wins?
5. What does high precision with low recall mean for an NER system?

6. **Exercise.** Run a pre-trained pipeline over a news paragraph and list every entity. Mark each as correct, wrong label or wrong boundary, and count how many entities the model missed.
7. **Exercise.** Write an EntityRuler with five patterns for the course names of your own college and place it before and after `ner`. Report any differences.
8. **Exercise.** Annotate the entities of three sentences by hand, run the model, and compute precision, recall and F1 using the formulas of Section 4.2.6.